

Code Review for AI-generated Code: Human-Centered Tooling

Ilya Zakharov*, Agnia Sergeyuk*

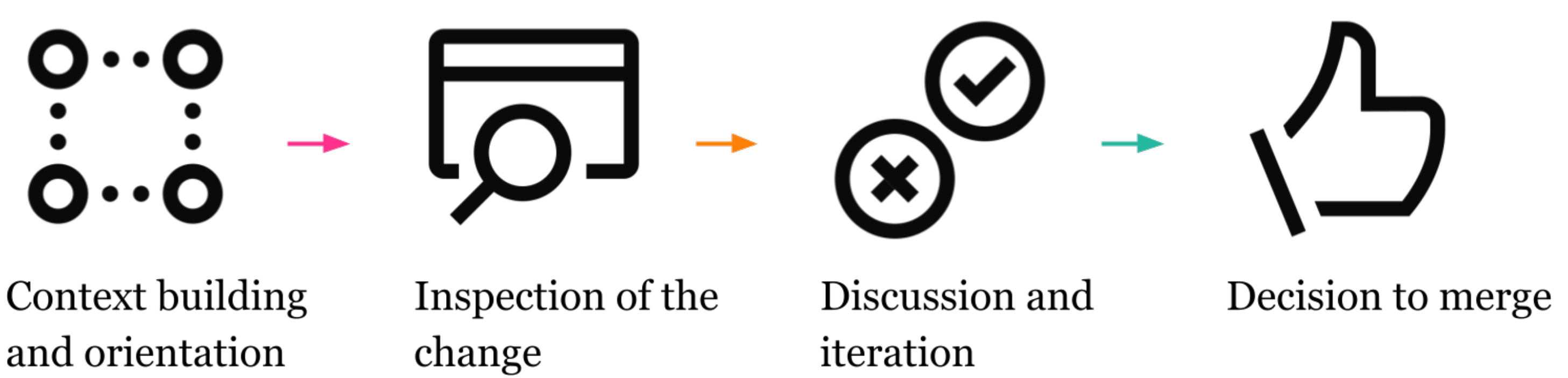
Background

As LLMs increasingly generate code, **humans remain responsible for reviewing it**. While substantial work exists on automating code review (LLMs as reviewers), we focus on human-AI collaboration: how to help human reviewers effectively assess LLM-generated code. We examine **augmentation over automation**, studying interfaces and workflows that improve human judgment of LLM outputs rather than replacing human reviewers entirely.

Consider an LLM-based agentic system (e.g., Junie) submitting changes across 10 files in your repository. How can we support the human reviewer in this context?

Code Review Framework

Based on existing literature, we conceptualize code review as a four-stage process [1, 2]:



Key Distinctions for AI-Generated Code Review

In AI-generated code review, the human reviewer often serves as a co-author, possessing deep knowledge of the codebase architecture and maintaining comprehensive mental models of system components. This dual role simplifies **context building and orientation phase** compared to traditional peer review scenarios.

The critical support needs emerge during **inspection and iteration phases**. Unlike conventional code review, where acceptance decisions are typically final, AI-generated code review involves iterative refinement cycles. Reviewers may accept suggestions conditionally, triggering further AI iterations, making the **decision process** more dynamic and provisional rather than binary.

This iterative nature fundamentally shifts the reviewer’s role **from gatekeeper to collaborator**, requiring new interface paradigms and workflow support tools.

Links



Read the full report



Contact the authors

Which 3 features would enhance your experience?

🔗 Context building and orientation

Help reviewers keep a mental model and set expectations:

- ▶ **State goals and summarize changes** — explain why the change was made and what was done.
- ▶ **Provide quick beacons** — highlight files/lines changed, libraries involved, and expected impact.
- ▶ **Offer deeper explanations** — give more detailed “why/what” context for less experienced reviewers.

🔍 Inspection of the change

Support reviewers in comparing expectations with actual code:

- ▶ **Use dynamic checklists and review templates** tailored to the type of change.
- ▶ **Enable two-phase inspection** — a high-level “scan” → detailed view.
- ▶ **Annotate code and cite sources** to justify chosen solutions.
- ▶ **Prioritize core revisions** — review important files and foundational changes first.
- ▶ **Group related changes** to reduce context switching and avoid duplication.
- ▶ **Provide a diff view and architecture visualization** — show edits, dependencies, and summaries of affected nodes.
- ▶ **Integrate static analysis** — include background checks for security, dependencies, and test results.
- ▶ **Check against up-to-date conventions** and best practices, enforcing automatically where possible.

💬 Discussion and iteration

Facilitate collaboration and improvement during review:

- ▶ **Present multiple solution options** and alternatives where relevant.
- ▶ **Enable per-line and change-level discussions** to clarify issues in context.
- ▶ **Maintain human gatekeeping** for architectural and functional decisions.

Instructions: Place stickers next to up to 3 features you’d find most valuable!

Key Takeaways

- ▶ **Augment, don’t replace:** tools should support human judgment.
- ▶ **Manage cognitive load:** scan → detail; chunk & prioritize.
- ▶ **Ground decisions:** annotate code, cite sources, link prompts.
- ▶ **Iterate with AI:** provisional acceptance → guided re-runs.

VOTE: What would make AI code review easier for you?

Bibliography

- [1] [Pavlína Wurzel Gonçalves et al.](#) “Code Review Comprehension: Reviewing Strategies Seen Through Code Comprehension Theories”. ()
- [2] [Lo Gullstrand Heander, Emma Söderberg, and Christofer Rydenfält.](#) “Code Review as Decision-Making–Building a Cognitive Model from the Questions Asked During Code Review”. ()

*Authors contributed equally. Corresponding author: agnia.sergeyuk@jetbrains.com